Solution Requirements

Date	15 March 2026
Team ID	XXXXXX
Project Name	LegalEase
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

The following requirements are derived from the supplied LegalEase project description and workflow. Functional requirements describe what the system should do, while non-functional requirements describe how the system should operate.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	The supplied LegalEase specification does not define end-user login or identity verification. The system does use a .env file for secure Gemini API-key configuration.	Low
2	Authorization levels	No separate user roles or permission levels are specified in the supplied project. The application is described as a user-facing document generation interface.	Low
3	External interfaces	Integrate the Google Gemini Generative AI service through the Gemini API/SDK. The FastAPI backend exposes POST /generate and exchanges document_type, parties, terms and dates.	High
4	Transactions processing	Process a document-generation request by validating the input, sending a structured prompt to Gemini, receiving generated legal text, and returning it to the frontend for preview, editing and export.	High
5	Reporting	Provide generated-document outputs and downloads in TXT, DOCX and PDF formats. The supplied specification does not define separate analytics dashboards or reporting modules.	Medium
6	Business rules, etc.	Generate structured legal documents from document type, involved parties, terms/conditions and effective date. Terms may be entered as semicolon-separated clauses and converted into structured terms/bullets.	High
7	Compliance to laws or regulations	Handle sensitive user data securely and keep the Gemini API key in environment configuration. No specific regulatory standard such as GDPR or HIPAA is explicitly specified in the supplied project.	Medium
8	Other	Support editable document preview, custom branding such as logos/fonts, formatted headings, terms tables, headers/footers, and multi-format document export.	High

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	The system should process valid document-generation requests through FastAPI and Gemini and return generated content to the Streamlit interface.	Successful response and document generation for valid inputs; exact response-time target is not specified in the supplied project.
2	Scalability	The architecture separates Streamlit frontend, FastAPI backend, Gemini AI integration and document-formatting modules, allowing the components to be deployed and scaled independently.	Application remains functional when the backend, frontend and AI integration are deployed as separate components; no numeric concurrency target is specified.
3	Security & Data Privacy	Sensitive configuration such as the Gemini API key should be stored in a .env file. The project also specifies secure handling of sensitive user data.	API key is not hard-coded in application input/output; environment configuration is used. No specific encryption standard is defined in the supplied project.
4	Reliability & Availability	The application should support end-to-end operation from user input through Gemini generation, preview/editing and document export.	End-to-end workflow completes successfully for supported input combinations; local deployment testing verifies the listed functions.
5	Usability & Accessibility	Provide a responsive Streamlit interface with clear inputs for document type, parties, terms and effective date, plus Generate, Edit and Download actions.	Users can enter all required fields, preview/edit generated content and download TXT/DOCX/PDF outputs without needing a legal background.
6	Other	Maintainability and portability are supported through Python, FastAPI, Streamlit, modular routes/AI-core/formatting utilities, requirements.txt and optional deployment configurations.	Project dependencies can be installed from requirements.txt and the application can be run locally using FastAPI/Uvicorn and Streamlit.

Note: Where the supplied LegalEase project does not define a numeric target (for example, exact response time, concurrent users, uptime percentage, encryption algorithm, or accessibility standard), the acceptance criteria above retain the project-defined behavior instead of inventing unsupported numeric specifications.